

Inteligência Artificial

Aplicação das estratégias de busca local

Prof. Chauã Queirolo | <https://github.com/chaaua>

1

Sumário

Introdução

Busca local

Decisões de projeto

Exemplos

Considerações finais

2

Elementos de um problema de busca

Estado inicial

Configuração de partida do problema

Ações

Operações que podem ser executadas em cada estado

Modelo de transição

Define qual estado resulta da aplicação de uma ação

Teste de objetivo

Verifica se o estado atual satisfaz a meta

Custo do caminho

Valor acumulado das ações executadas

Solução

Sequência de ações que levam do **estado inicial** para o **estado objetivo**

Solução ótima

Solução com o menor **custo de caminho**

Busca local

- Mantém apenas o estado atual
- Explora estados vizinhos
- Usa função de avaliação
- Melhora iterativamente a solução
- Não armazena a árvore de busca

Busca local

A mesma técnica pode:

- convergir rápido em um problema
- estagnar em outro
- ficar presa em ótimos locais
- ser sensível ao estado inicial

Por que a mesma técnica funciona bem em um problema e mal em outro?

- O algoritmo não é o único fator
- O desempenho depende da modelagem do problema

qualidade da modelagem > **qualidade da implementação isolada**

7

Decisões de projeto

1. Representar a solução

- como codificar um estado?
- como respeitar restrições?

3. Medir qualidade

- o que significa "melhor"
- como penalizar violações

2. Gerar vizinhos

- quais mudanças locais são permitidas?
- qual o custo por iteração?

4. Escolher o movimento

- melhor vizinho?
- vizinho aleatório?
- movimento não tabu?

8

Representação da Solução

9

Representação da solução

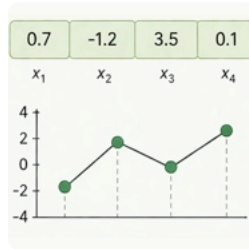
- Uma representação ruim pode inviabilizar a busca
- Boas representações:
 - fáceis de manipular
 - geram vizinhos rapidamente
 - refletem as restrições do problema
 - facilitam avaliar a solução

10

Exemplos comuns



Permutação

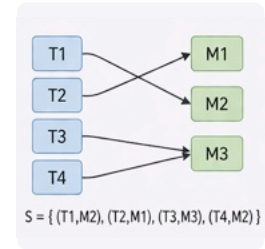


Vetor de valores

	H1	H2	H3	H4
T1	1	0	0	0
T2	0	1	0	1
T3	0	0	1	0

(1 = alocado, 0 = não alocado)

Matriz de valores



Conjunto de pares

Vizinhança

Vizinhança

A vizinhança precisa equilibrar exploração e eficiência.

Pequena demais ▼

- . progresso lento
- . pouca exploração

Grande demais ▲

- . custo alto por iteração
- . seleção mais cara

Operadores típicos

Troca

Inserção

Inversão

Deslocamento

Reatribuição

Qualidade

15

Função de avaliação

Define o que é "melhor"

Se a função estiver errada, o algoritmo melhora a métrica errada

Qualidade = **objetivo principal** + **penalidades**

boa métrica → boa direção de busca

16

Exemplos de termos na função

- Número de conflitos
- Distância total
- Atraso acumulado
- Penalidade por restrição violada

Movimento

Hill Climbing

Funciona bem quando

- a paisagem é mais suave
- a vizinhança é bem escolhida
- há boa solução inicial
- a função de avaliação é informativa

Funciona mal quando

- há muitos ótimos locais
- os platôs são frequentes
- o espaço é muito acidentado
- a busca depende demais do ponto inicial

Busca Tabu

- **Busca Tabu = busca local + memória**
- Útil quando a busca insiste em voltar para onde já esteve
 - Evita ciclos curtos
 - Força novas regiões da busca
 - Reduz repetição de padrões
 - Depende do tamanho da lista

Simulated Annealing

- Amplia a exploração no início e estabiliza no final
 - Aceita sempre soluções melhores
 - Às vezes aceita soluções piores
- **temperatura alta:** mais exploração
- **temperatura baixa:** mais estabilidade
- Aceitar uma piora agora pode permitir uma melhora maior depois

21



Inicialização

22

Parâmetros

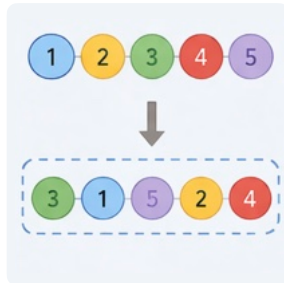
Parâmetro ruim pode mascarar o potencial do algoritmo

- Iterações máximas
- Tamanho da vizinhança
- Reinícios aleatórios
- Tamanho da lista tabu
- Temperatura inicial
- Taxa de resfriamento

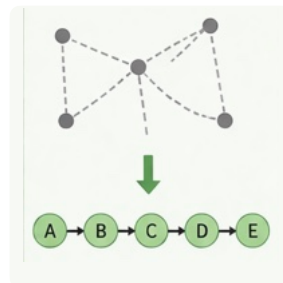
Estado inicial

- Estado inicial é um componente de projeto, não um detalhe
- O estado inicial influencia na:
 - velocidade
 - qualidade da solução
 - estabilidade entre execuções

Estado inicial



Aleatório
mais diversidade
mais variabilidade



Heurístico
arranque melhor
convergência rápida



Múltiplos reinícios
mais robustez
melhor cobertura

25

Heurística

Heurística é qualquer conhecimento que torne a busca mais inteligente

Estado inicial

- . construção gulosa
- . priorizar casos críticos

Seleção de vizinhos

- . ordenar movimentos
- . amostrar vizinhos promissores

Função de custo

- . termos de qualidade
- . penalidades calibradas

Reinícios e desempates

- . quando reiniciar
- . como desempatar movimentos

26

Falhas

Ótimo local

para cedo demais

nenhum vizinho melhora

Oscilação

vai e volta entre poucas soluções

vizinhança mal calibrada

Platô

muitos estados equivalentes

falta direção clara

Estagnação

melhora no início e depois trava

parâmetros inadequados

Exemplos

Exemplo completo 1: escalonamento

- Estado: ordem das tarefas nas máquinas
- Vizinhos: trocar, inserir ou inverter tarefas
- Avaliação: atraso total + ociosidade + penalidades
- Heurística inicial: priorizar prazo ou duração
- Risco típico: muitos ótimos locais.

Exemplo completo 1: escalonamento

- Estado: ordem das tarefas nas máquinas
- Vizinhos: trocar, inserir ou inverter tarefas
- Avaliação: atraso total + ociosidade + penalidades
- Heurística inicial: priorizar prazo ou duração
- Risco típico: muitos ótimos locais.

Roteamento de entregas

Problema: definir a ordem de visita aos pontos de entrega minimizando custo total

Estado: sequência das paradas da rota [Depósito, A, C, B, D, E, Depósito]

Vizinhança:

- troca de duas paradas
- inserção de uma parada em outra posição
- inversão de um trecho
- deslocamento de uma parada

Função de avaliação:

- distância total
- tempo total
- penalidade por atraso
- penalidade por retorno desnecessário

Heurísticas:

- aproximar pontos geograficamente próximos
- penalizar cruzamentos de rota
- favorecer agrupamentos regionais

Grade horária de aulas

Problema: alocar turmas em horários e salas minimizando conflitos

Estado: matriz com turma, horário e sala

Turma 1 → segunda 19h sala A

Turma 2 → terça 21h sala B

Vizinhança:

- trocar horários de duas turmas
- mudar uma turma de sala
- deslocar uma turma para outro horário
- reatribuir professor ou sala

Função de avaliação:

- número de conflitos
- penalidade por sala inadequada
- penalidade por janelas ruins
- penalidade por professor em horários impossíveis

Heurísticas:

- alocar primeiro turmas mais restritas
- penalizar conflitos de professor
- privilegiar salas compatíveis com capacidade

Alocação de tarefas em máquinas

Problema: distribuir tarefas entre máquinas minimizando tempo total ou balanceando carga

Estado: conjunto de pares tarefa → máquina ou lista de tarefas por máquina

M1 = [T1, T4]

M2 = [T2, T3, T5]

Vizinhança:

- mover uma tarefa para outra máquina
- trocar tarefas entre máquinas
- reordenar tarefas dentro da mesma máquina
- redistribuição de tarefa

Função de avaliação:

- tempo ocioso
- desbalanceamento de carga
- penalidade por incompatibilidade de recurso

Heurísticas:

- enviar tarefas longas para máquinas menos carregadas
- evitar concentrar tarefas pesadas em uma única máquina
- penalizar alocações inviáveis

Seleção de características em aprendizado de máquina

Problema: escolher subconjunto de atributos que melhora desempenho do modelo

Estado: vetor binário indicando atributos escolhidos

[1, 0, 1, 1, 0, 0, 1]

Vizinhança:

- ativar um atributo
- desativar um atributo
- trocar um atributo selecionado por outro não selecionado

Função de avaliação:

- acurácia
- F1-score
- erro de validação
- penalidade por número de atributos

Heurísticas:

- privilegiar atributos mais informativos
- penalizar redundância
- favorecer subconjuntos menores com bom desempenho

Configuração de parâmetros de um sistema

Problema: ajustar parâmetros para maximizar desempenho ou minimizar erro

Estado: vetor de valores reais ou discretos

[taxa_aprendizado=0.01, lambda=0.1, profundidade=6]

Vizinhança:

- aumentar ou diminuir um parâmetro
- alterar dois parâmetros simultaneamente
- substituir um valor por outro próximo

Função de avaliação:

- erro
- acurácia
- custo computacional
- penalidade por complexidade

Heurísticas:

- explorar primeiro regiões promissoras
- usar passos menores em fases finais
- penalizar configurações muito caras

Caixeiro Viajante

Problema: Caixeiro viajante precisa visitar n cidades, sem repetir nenhuma

Estado: vetor de valores discretos

[A, C, E, B, D]

Vizinhança:

- Troca
- Inserção
- Rotação
- Inversão

Função de avaliação:

- distância total

Heurísticas:

- explorar primeiro regiões promissoras
- usar passos menores em fases finais
- penalizar configurações muito caras

Busca local

37

Otimizar a busca local

- Melhorar busca local é um processo iterativo de engenharia
 - melhorar a representação
 - redefinir a vizinhança
 - recalibrar a função de avaliação
 - usar reinícios
 - incorporar memória
 - aceitar pioras controladas
 - combinar heurística construtiva + busca local

38

Comparação Experimental

- **Qualidade final**
 - melhor, média e pior solução
- **Tempo**
 - tempo até boa solução e tempo total
- **Estabilidade**
 - variação entre execuções
- **Sensibilidade**
 - efeito do estado inicial e parâmetros
- **Robustez**
 - capacidade de escapar de estagnação

39

Métricas

- **Qualidade da solução**
 - melhor valor
 - média entre execuções
- **Tempo**
 - até primeira boa solução
 - tempo total
- **Estabilidade**
 - desvio entre execuções
 - pior caso
- **Dinâmica da busca**
 - iterações sem melhora
 - taxa de convergência

40

Erros comuns

- **Função de custo ruim**

- não reflete o objetivo
- penalidades desequilibradas

- **Vizinhança inadequada**

- muda pouco a solução
- custa caro demais

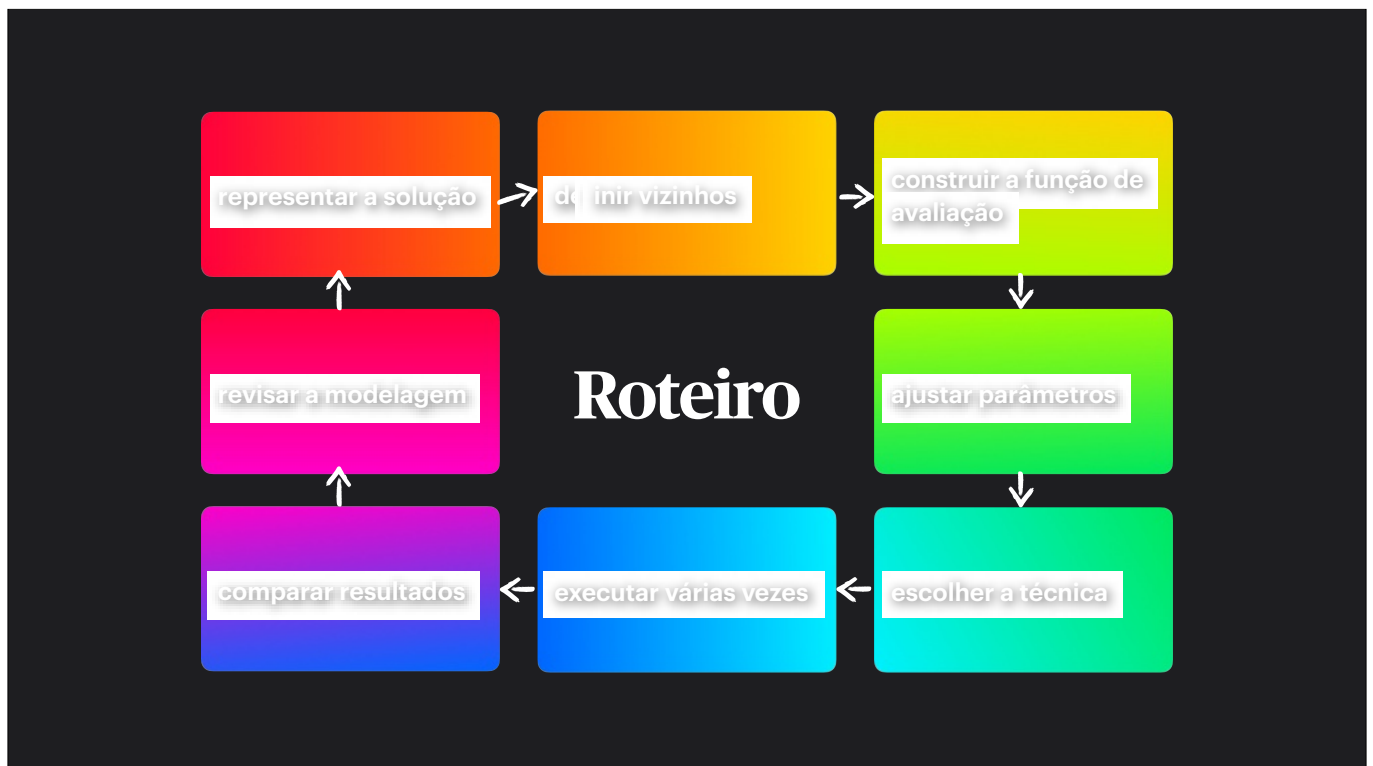
- **Parâmetros arbitrários**

- sem ajuste
- sem análise de sensibilidade

- **Experimento fraco**

- poucas execuções
- comparação injusta

41



42

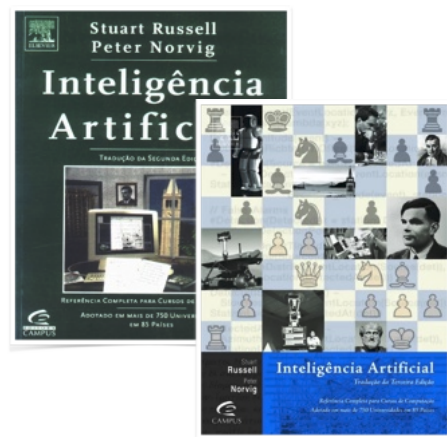
Considerações Finais

- O algoritmo sozinho não basta
- Modelagem define a qualidade da busca
- Heurísticas e parâmetros são decisivos
- Comparar exige várias execuções
- Melhorar busca local é engenharia

43

Referências Bibliográficas

- S. J. Russell & P. Norvig. **Artificial Intelligence: A Modern Approach**. Prentice Hall, 3rd edition, 2010.



44